Contender 20: Second-Order Diagonal Hyperreflection in BusyLean

A constructive type-level jump beyond the large-Veblen contender

Research proposal for the BusyLean project

13 October 2026

Abstract

BusyLean is a Lean 4 version of the "name the biggest number" contest: every entry is a closed computable natural number and every victory claim is a formal strict inequality. At the audited baseline, Contender 19 combines a large-Veblen-length ladder of strongly normalizing object languages with a natural-rank hierarchy of *transformer lifts*. This article proposes Contender 20, whose main novelty is not a longer ordinal notation and not one more numerical iteration, but a jump in the *type of the amplifier*.

We introduce a *hyperlift*, an endomorphism of transformer lifts. We define countable splicing of lifts, W-tree traversal by an arbitrary hyperlift, an autonomous second-order lift, and a diagonal hierarchy of hyperlifts. The proofs isolate a reusable invariant: a good lift both preserves progressive transformers and strictly exceeds its input transformer. The second-order hierarchy preserves good lifts and strictly exceeds its base lift, without any monotonicity assumption on oracle reflection.

At each input n, the new function first embeds a complete first-order hyperreflection layer of rank $\mathrm{family}_{19}(n) + 1$ and then applies a second-order diagonal hierarchy of the same rank. The formal comparison is factored through the explicit chain

$$\mathrm{family}_{19}(n) < \mathrm{firstOrder}_{20}(n) < F_{20}(n).$$

Consequently the proposed closed number

$$\mathrm{contender}_{20} = F_{20}(F_{20}(42))$$

strictly exceeds Contender 19. All new definitions are structural and computable. The package includes Lean source, comparison theorems, a repository patch, a smoke test, a static constructivity audit, and this compiled article. The LaTeX and static audits were executed; a Lean 4.34.0 kernel build remains a required pre-merge step because that toolchain was unavailable in the package generation environment.

Contents

1 Problem, baseline, and result **3**

1.1 The contest rule 3

1.2 Audited repository state 4

1.3 Proposed result 4

2 Why a type-level jump is the right next move **5**
2.1 Trivial and near-trivial victories . . . 5
2.2 A longer ordinal ladder . . . 5
2.3 Reflection over Lean itself . . . 5
2.4 The chosen principle . . . 5

3 The inherited algebra **5**
3.1 Families and transformers . . . 6
3.2 Transformer lifts . . . 6
3.3 Tree stages and first-order diagonalization . . . 6

4 Second-order definitions **7**
4.1 Hyperlifts . . . 7
4.2 Splicing lifts . . . 7
4.3 Tree traversal by a hyperlift . . . 7
4.4 The autonomous second-order lift . . . 7
4.5 Second-order diagonal ranks . . . 8

5 The invariant: good lifts **8**
5.1 Definition . . . 8
5.2 Splice closure . . . 8
5.3 The rank-zero hyperlift . . . 9
5.4 Tree-stage closure . . . 9
5.5 Diagonal hierarchy . . . 9

6 Definition of Contender 20 **10**
6.1 The predecessor family . . . 10
6.2 Schedule . . . 10
6.3 Rank and embedded first-order layer . . . 11
6.4 Second-order closure . . . 11

7 Formal dominance chain **11**
7.1 The base lift is good . . . 11
7.2 First strict step . . . 12
7.3 Second strict step . . . 12
7.4 Pointwise predecessor domination . . . 12
7.5 Closed contender comparison . . . 12

8 Proof dependency graph **13**

9 Computability and foundational hygiene **13**
9.1 Structural recursion . . . 13
9.2 Forbidden mechanisms . . . 13
9.3 No oracle monotonicity assumption . . . 14

10 How large is the increase? **14**
10.1 What is formally established . . . 14
10.2 Why this is more than another natural rank . . . 14
10.3 Why no exact ordinal calibration is claimed . . . 15

10.4 A useful lower-bound reading . . . 15

11 Lean engineering considerations **15**
11.1 Avoiding enormous reduction . . . 15
11.2 Expected compilation profile . . . 16
11.3 Potential elaboration sensitivities . . . 16

12 Integration and validation **16**
12.1 Files to add . . . 16
12.2 Required build commands . . . 16
12.3 Validation actually completed . . . 17

13 Further research **17**
13.1 Third and higher operator levels . . . 17
13.2 Transfinite hyper-ranks . . . 17
13.3 Combining operator order with stronger ordinal atoms . . . 17
13.4 Ordinal calibration . . . 18
13.5 A certified Lean-kernel reflection contender . . . 18
13.6 Performance instrumentation . . . 18

14 Conclusion **18**

A Public API summary **19**

B Static audit scope **19**

C Selected references **20**

1 Problem, baseline, and result

1.1 The contest rule

The governing principle is stronger than merely writing an expression that is obviously enormous. A valid contender must satisfy three requirements.

1. It is a closed Lean term of type `Nat`.
2. It is computable in principle: no appeal to a Busy Beaver, an unbounded search whose termination is postulated, or classical choice.
3. Its superiority over the incumbent is certified by a theorem accepted by Lean's kernel.

The values in question must never be evaluated. Their magnitude is established extensionally, through generic strict-growth lemmas whose proof terms are small. That distinction—huge definitions but small comparison proofs—is central to BusyLean's engineering.

1.2 Audited repository state

This package was designed against the BusyLean `main` tree at commit

`64d31a0cf6f1aa4477a088be5554957de6e87a61`.

The repository pins Lean `v4.34.0`. Its README names Contender 19 as the current entry. In condensed form, the incumbent is

```
def family18 (n : Nat) : Nat := F18 (F18 n)
def F19 (n : Nat) : Nat :=
  diagonalLift (schedule19 n) (family18 n + 1)
    lTransformer (fun _ => family18) n n
def contender_19 : Nat := F19 (F19 42)
```

The object-language component, `lTransformer`, reflects programs in a ladder whose atoms are Schütte-style large-Veblen terms. The amplifier component, `diagonalLift`, is a hierarchy of endomorphisms of transformers. The repository proves

```
theorem contender_18_lt_contender_19 :
  contender_18 < contender_19
```

and, more strongly, pointwise domination of the entire predecessor family.

1.3 Proposed result

The proposal adds one new amplification level:

Level	Carrier	Intended action
0	oracle families	supply numerical data to reflected programs
1	transformers	map oracle families to stronger families
2	transformer lifts	map transformers to stronger transformers
3	hyperlifts	map whole transformer-lift hierarchies to stronger ones

Contender 19 reaches Level 2. Contender 20 diagonalizes at Level 3. Its key formal statements are

```
theorem family19_lt_firstOrder20 (n : Nat) :
  family19 n < firstOrder20 n

theorem firstOrder20_lt_F20 (n : Nat) :
  firstOrder20 n < F20 n

theorem contender_19_lt_contender_20 :
  contender_19 < contender_20
```

The intermediate `firstOrder20` is deliberate. It makes the structural margin visible inside the formal proof: before the second-order hierarchy does anything, the construction has already performed a complete first-order `diagonalLift` using a predecessor-sized schedule and rank.

2 Why a type-level jump is the right next move

Several simpler definitions would win the literal comparison but would not be serious successors.

2.1 Trivial and near-trivial victories

The definitions

$$\text{contender}_{19} + 1, \qquad F_{19}(F_{19}(F_{19}(42))), \qquad \text{or} \qquad F_{19}^{k}(42)$$

for some fixed k are computable and easy to compare. They add no new mathematical mechanism. BusyLean's accepted sequence has consistently sought a new language, a new reflection principle, a longer well-founded universe ladder, or a new operator-level closure. Merely inserting another numeral or another fixed composition would be contrary to that trajectory.

2.2 A longer ordinal ladder

A natural alternative is to replace the large-Veblen atom order by a stronger notation system—for example, a collapsing-function system associated with a Bachmann–Howard or stronger ordinal. That direction remains important, but it has a high formal cost. One must define normal forms, a decidable comparison, a finite enumerator, and a direct well-foundedness proof strong enough for `FL.wfUniv`. The current large-Veblen development is already a substantial proof-engineering achievement. A rushed collapsing system would create much more proof risk than the present proposal.

Moreover, an ordinal extension changes the *length of the object-language ladder* while leaving the amplifier type unchanged. The present construction is orthogonal: it closes over the entire amplifier used by Contender 19. Eventually the two ideas should be combined.

2.3 Reflection over Lean itself

One can imagine enumerating kernel-checked Lean terms and taking a bounded maximum. Such a construction could be exceptionally strong, but a trustworthy version requires a reflected Lean kernel, a certified evaluator, a finite coding discipline, and careful avoidance of self-reference. That is a separate research program, not an incremental repository patch.

2.4 The chosen principle

Contender 19's `diagonalLift` already treats a transformer lift as an object that can be iterated and diagonally reused. The most canonical next step is therefore to treat *that whole lift* as data for a higher-order operator. The resulting code is compact because the generic proof architecture can be raised one level almost verbatim. The increase is nevertheless structural: the carrier changes from

$$\mathsf{Transformer} \to \mathsf{Transformer}$$

to

$$(\mathsf{Transformer} \to \mathsf{Transformer}) \to (\mathsf{Transformer} \to \mathsf{Transformer}).$$

3 The inherited algebra

This section restates the existing abstractions needed by the proposal. The exact Lean definitions are in `MetaReflection.lean`, `HyperReflection.lean`, and their facts modules.

3.1 Families and transformers

A family is a countable two-argument array of natural numbers,

$$\mathsf{Family} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

A transformer maps one family to another,

$$\mathsf{Transformer} = \mathsf{Family} \to \mathsf{Family}.$$

The first argument of a family is a visible coordinate; the second is the main numerical input.

Definition 3.1 (Progressive transformer)**.** A transformer T is *progressive* when every visible value of its input family is strictly below every output coordinate at the same numerical input:

$$\mathsf{Progressive}(T) :\Longleftrightarrow \forall s, i, o, n,\ i \le n \implies s(i, n) < T(s)(o, n).$$

This property is stronger than the one inequality needed for a contender, and that strength is what makes it compositional.

3.2 Transformer lifts

A transformer lift is an endomorphism of transformers,

$$\mathsf{TransformerLift} = \mathsf{Transformer} \to \mathsf{Transformer}.$$

Two predicates govern a lift L.

$$\begin{aligned}
\mathsf{PreservesProgressive}(L) &:\Longleftrightarrow \forall T,\ \mathsf{Progressive}(T) \implies \mathsf{Progressive}(L(T)),\\
\mathsf{StrictlyInflationary}(L) &:\Longleftrightarrow \forall T,\ \mathsf{Progressive}(T) \implies\\
&\qquad \forall s, i, o, n,\ i \le n \implies T(s)(i, n) < L(T)(s)(o, n).
\end{aligned}$$

The strict predicate is intentionally pointwise and coordinate-flexible. It does not assert that L is monotone as a function of T or of the oracle family. Oracle programs may branch on numerical answers, so such monotonicity would be false in general.

3.3 Tree stages and first-order diagonalization

Given a local lift L and a base transformer T, the existing `stageLift L T a` traverses a countably branching well-founded tree a. At a leaf it applies L to T; at a node it recursively constructs one child transformer per coordinate, splices the children, and applies L to the splice.

For a fixed schedule a, `diagonalLift a k` is a natural-rank hierarchy:

$$\begin{aligned}
D_0(T) &= \mathsf{liftTransformer}(T),\\
D_{k+1}(T) &= \mathsf{stageLift}(D_k, D_k(T), a).
\end{aligned}$$

The same lower rank D_k appears twice at a successor: as the local lift and as the first amplification of the base. This is the first-order diagonal pattern that the proposal raises one type level.

4 Second-order definitions

4.1 Hyperlifts

Definition 4.1 (Hyperlift)**.** A hyperlift is an endomorphism of transformer lifts:

$$\mathsf{HyperLift} = \mathsf{TransformerLift} \to \mathsf{TransformerLift}.$$

In Lean:

```
abbrev HyperLift := TransformerLift → TransformerLift
```

The name is descriptive rather than foundational: no claim is made that this is a canonical hierarchy in proof theory. It is simply the next endomorphism level required by the existing design.

4.2 Splicing lifts

Existing `spliceTransformers` lets output coordinate i choose transformer i. The second-order analogue is pointwise in the supplied base transformer:

```
def spliceLifts (ops : Nat → TransformerLift) : TransformerLift :=
  fun base => spliceTransformers (fun i => ops i base)
```

Thus, after a base transformer T is supplied,

$$\mathsf{spliceLifts}(L_0, L_1, \ldots)(T)$$

is the splice of $L_0(T), L_1(T), \ldots$. Coordinate i selects $L_i(T)$ and uses that transformer's canonical output coordinate zero.

4.3 Tree traversal by a hyperlift

```
def stageHyperLift (hyper : HyperLift) (base : TransformerLift) :
    WTree Nat → TransformerLift
  | .leaf => hyper base
  | .node f =>
      hyper (spliceLifts (fun i => stageHyperLift hyper base (f i)))
```

This is exactly the same W-tree pattern one type level higher. Structural recursion on `WTree Nat` supplies computability. No search, quotient choice, or semantic oracle is introduced.

4.4 The autonomous second-order lift

A naive type-level analogue of autonomous lifting would be $L \mapsto (T \mapsto \mathsf{stageLift}(L, T, \mathsf{index}))$. For the comparison proof we want the result to strictly exceed L itself, not merely the base transformer T. We therefore diagonally seed the stage with $L(T)$:

```
def liftLift (lift : TransformerLift) : TransformerLift :=
  fun base => stageLift lift (lift base) index
```

For a good L, two strict steps are now visible:

$$T < L(T) < \mathsf{stageLift}(L, L(T), \mathsf{index}).$$

The first is strict inflationarity of L; the second is the existing `base_lt_stageLift` theorem.

4.5 Second-order diagonal ranks

```
def diagonalHyperLift (schedule : WTree Nat) : Nat → HyperLift
  | 0, base => liftLift base
  | k + 1, base =>
      stageHyperLift (diagonalHyperLift schedule k)
        (diagonalHyperLift schedule k base) schedule
```

Writing H_k for `diagonalHyperLift schedule k`,

$$H_0(L) = \mathsf{liftLift}(L),$$
$$H_{k+1}(L) = \mathsf{stageHyperLift}(H_k, H_k(L), \mathsf{schedule}).$$

At each successor rank the preceding hyper-rank appears extensionally as the local operator and intensionally in the seed. The recurrence is a literal second-order analogue of `diagonalLift`.

5 The invariant: good lifts

5.1 Definition

Definition 5.1 (Good transformer lift)**.** A transformer lift is good when it both preserves progressive transformers and strictly exceeds its input transformer:

```
def GoodLift (lift : TransformerLift) : Prop :=
  PreservesProgressive lift ∧ StrictlyInflationary lift
```

For a hyperlift H we use two properties.

$$\mathsf{PreservesGoodLifts}(H) :\Longleftrightarrow \forall L,\ \mathsf{GoodLift}(L) \implies \mathsf{GoodLift}(H(L)),$$
$$\mathsf{StrictlyLiftInflationary}(H) :\Longleftrightarrow \forall L, T,\ \mathsf{GoodLift}(L) \wedge \mathsf{Progressive}(T) \implies \forall s, i, o, n,\ i \le n \implies L(T)(s)(i, n) < H(L)(T)(s)(o, n).$$

The second property is the exact higher-order analogue of strict inflationarity. It compares two lifts after applying both to the same progressive base transformer.

5.2 Splice closure

Lemma 5.2 (Splicing preserves goodness)**.** *If every L_i is good, then `spliceLifts (fun i => L_i)` is good.*

Proof architecture. For progressivity preservation, apply each L_i to a progressive base transformer and invoke the existing `spliceTransformers_progressive`. For strict inflationarity, output coordinate o selects L_o. Its strictness compares the base transformer's visible coordinate i with canonical output coordinate zero of $L_o(T)$. This is definitionally the selected splice value. □

No comparison between different L_i is needed.

5.3 The rank-zero hyperlift

Theorem 5.3 (`liftLift_preservesGoodLifts`)**.** *The autonomous second-order lift maps good lifts to good lifts.*

Proof architecture. Let L be good and T progressive. Then $L(T)$ is progressive. The existing `stageLift_progressive` theorem shows that

$$\mathsf{stageLift}(L, L(T), \mathsf{index})$$

is progressive. For strict inflationarity, chain

$$T(s)(i,n) < L(T)(s)(i,n) < \mathsf{stageLift}(L, L(T), \mathsf{index})(s)(o,n).$$

The first inequality uses strictness of L and the second uses `base_lt_stageLift` with base $L(T)$. □

Theorem 5.4 (`liftLift_strictlyLiftInflationary`)**.** *The autonomous second-order lift strictly exceeds its input lift.*

Proof architecture. For good L and progressive T, apply `base_lt_stageLift` directly to the base transformer $L(T)$. The target is definitionally `liftLift L T`. □

5.4 Tree-stage closure

Theorem 5.5 (`stageHyperLift_good`)**.** *If a hyperlift preserves good lifts and its base lift is good, then every W-tree stage is good.*

Proof architecture. Induct structurally on the W-tree. At a leaf, apply the preservation hypothesis. At a node, recursively obtain a good lift for each child, splice them using the splice lemma, and apply the preservation hypothesis to the splice. □

Theorem 5.6 (`base_lt_stageHyperLift`)**.** *If a hyperlift preserves good lifts and strictly exceeds its input lift, then every W-tree stage strictly exceeds its base lift.*

Proof architecture. Induct on the W-tree. The leaf case is the strictness hypothesis. At a node, first follow child zero, obtaining

$$L(T)(s)(i,n) < \mathsf{child}_0(T)(s)(0,n).$$

The child lifts are good, so their splice is good. Apply hyperlift strictness to the splice at visible coordinate zero. By the definition of `spliceLifts`, that visible coordinate is exactly child zero. The two strict inequalities compose. □

The explicit child-zero witness is important. It avoids any need to prove that the hyperlift is monotone in its input lift.

5.5 Diagonal hierarchy

Theorem 5.7 (Preservation at every second-order rank)**.** *The declaration* ***`diagonalHyperLift_preservesGoodLifts`*** *states that every natural rank of* ***`diagonalHyperLift`*** *preserves good lifts.*

Proof architecture. Induct on the rank. Rank zero is `liftLift_preservesGoodLifts`. At a successor, the induction hypothesis makes $H_k(L)$ good; then `stageHyperLift_good` applies with local hyperlift H_k and base lift $H_k(L)$. □

Theorem 5.8 (Strict growth at every second-order rank)**.** *The declaration* `diagonalHyperLift_strictlyLiftInflationary` *states that every natural rank of* `diagonalHyperLift` *strictly exceeds its original base lift.*

Proof architecture. Again induct on the rank. Rank zero is the strict theorem for `liftLift`. At a successor, first use the induction hypothesis to obtain

$$L(T) < H_k(L)(T).$$

The preservation theorem makes $H_k(L)$ good. Then `base_lt_stageHyperLift` gives

$$H_k(L)(T) < H_{k+1}(L)(T).$$

Compose the inequalities. □

A one-step rank theorem is extracted as a separate public lemma:

```
theorem diagonalHyperLift_rank_lt_succ ... :
  diagonalHyperLift schedule k base op seed i n <
    diagonalHyperLift schedule (k + 1) base op seed o n
```

This theorem is not required for the final contender comparison, but it exposes the internal strict chain across all predecessor-sized ranks.

6 Definition of Contender 20

6.1 The predecessor family

As in previous entries, the closed predecessor number is converted back into a pointwise family:

```
def family19 (n : Nat) : Nat := F19 (F19 n)
```

Definitionally,

$$\text{family}_{19}(42) = \text{contender}_{19}\,.$$

Using a family rather than the closed constant avoids hard-coding the incumbent and yields a stronger pointwise theorem.

6.2 Schedule

```
def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule19 n)))
```

The new schedule is not merely `schedule19 n`. It starts from the encoded Contender 19 schedule and applies a predecessor-sized number of higher `Autonomous.hyper` derivatives. These trees are syntax used as schedules; no semantic order comparison or quotient-level ordinal identity is required.

6.3 Rank and embedded first-order layer

```
def rank20 (n : Nat) : Nat := family19 n + 1

def baseLift20 (n : Nat) : TransformerLift :=
  diagonalLift (schedule20 n) (rank20 n)

def firstOrder20 (n : Nat) : Nat :=
  baseLift20 n lTransformer (fun _ => family19) n n
```

Thus `firstOrder20 n` is already a complete first-order diagonal hyperreflection using

- the large-Veblen reflection transformer `lTransformer`,
- the predecessor family as its packed oracle seed,
- a predecessor-sized `Autonomous.hyper` closure constructed from Contender 19's schedule, and
- a rank strictly larger than `family19 n`.

This definition is the formal witness separating a serious construction from a syntactic victory.

6.4 Second-order closure

```
def F20 (n : Nat) : Nat :=
  diagonalHyperLift (schedule20 n) (rank20 n)
    (baseLift20 n) lTransformer (fun _ => family19) n n

def contender_20 : Nat := F20 (F20 42)
```

There are two nested diagonal hierarchies of the same predecessor-controlled rank:

1. `baseLift20 n` is a first-order hierarchy of transformer lifts;
2. `diagonalHyperLift ... (baseLift20 n)` is a second-order hierarchy of hyperlifts acting on that complete first-order hierarchy.

The outer self-application in `contender_20` follows the established BusyLean convention and provides a clean final comparison step.

7 Formal dominance chain

7.1 The base lift is good

The existing first-order theorems immediately package the embedded lift:

```
theorem baseLift20_good (n : Nat) : GoodLift (baseLift20 n) :=
  ⟨diagonalLift_preservesProgressive (schedule20 n) (rank20 n),
    diagonalLift_strictlyInflationary (schedule20 n) (rank20 n)⟩
```

This small theorem is the bridge from the existing repository to the new second-order theory.

7.2 First strict step

Because a good lift preserves progressivity, applying `baseLift20 n` to the progressive `lTransformer` yields a progressive transformer. Feed it the constant family of `family19` coordinates and select visible coordinate zero:

Theorem 7.1 (`family19_lt_firstOrder20`)**.** *For every n,*

$$\mathrm{family}_{19}(n) < \mathrm{firstOrder}_{20}(n).$$

The proof is a direct application of progressivity. It does not inspect any large numerical value.

7.3 Second strict step

Apply the generic second-order strictness theorem to `baseLift20 n` and `lTransformer`:

Theorem 7.2 (`firstOrder20_lt_F20`)**.** *For every n,*

$$\mathrm{firstOrder}_{20}(n) < F_{20}(n).$$

This is the essential new inequality. Its left side is an entire predecessor-sized first-order hierarchy, not merely the predecessor family.

7.4 Pointwise predecessor domination

Transitivity gives

Theorem 7.3 (`family19_lt_F20`)**.** *For every n,*

$$\mathrm{family}_{19}(n) < F_{20}(n).$$

The repository already proves $n < F_{19}(n)$, so

$$n < F_{19}(n) < F_{19}(F_{19}(n)) = \mathrm{family}_{19}(n) < F_{20}(n).$$

Hence $n < F_{20}(n)$ for every n.

7.5 Closed contender comparison

At $n = 42$,

$$\mathrm{contender}_{19} = \mathrm{family}_{19}(42) < F_{20}(42).$$

Applying the input-growth theorem at $F_{20}(42)$ yields

$$F_{20}(42) < F_{20}(F_{20}(42)) = \mathrm{contender}_{20}\,.$$

Therefore:

Theorem 7.4 (`contender_19_lt_contender_20`)**.**

$$\mathrm{contender}_{19} < \mathrm{contender}_{20}\,.$$

The Lean proof is only one transitivity step:

```
theorem contender_19_lt_contender_20 : contender_19 < contender_20 :=
  Nat.lt_trans (family19_lt_F20 42) (input_lt_F20 (F20 42))
```

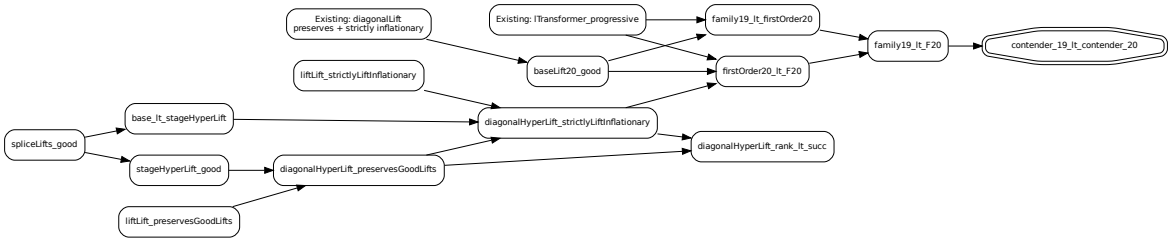


Figure 1: Dependency structure of the generic second-order facts and the final Contender 20 comparison. Existing BusyLean theorems enter only through the two leftmost nodes.

8 Proof dependency graph

The graph highlights a useful separation of concerns.

- The generic second-order library knows nothing about large-Veblen terms, `F19`, or the closed contender.
- The contender module supplies only schedule, rank, base lift, and seed.
- The comparison module connects the generic facts to the existing `lTransformer_progressive` theorem.

This modularity is important for future reuse. Any progressive reflection transformer can replace `lTransformer`, and any good transformer lift can replace `baseLift20`.

9 Computability and foundational hygiene

9.1 Structural recursion

Every new computational definition recurses on an inductive datum:

Definition	Recursion argument	Reason for termination
`stageHyperLift`	`WTree Nat`	structural recursion on the tree
`diagonalHyperLift`	`Nat`	structural recursion on the rank
`schedule20`	inherited constructors	composition of existing total functions
`F20`	none	finite composition of total definitions

Countable branching does not threaten computability. A W-tree node stores a function producing a child when a coordinate is requested; an evaluation uses only the finitely many coordinates demanded by its finite computation.

9.2 Forbidden mechanisms

The supplied source-hygiene audit strips comments and rejects executable occurrences of

`sorry`, `admit`, `axiom`, `noncomputable`, `unsafe`, `partial`, `opaque`, `extern`, `implemented_by`, and `Classical.choice`.

The recorded audit result is `PASS`. This is not a substitute for a kernel build, but it documents the intended constructive discipline.

9.3 No oracle monotonicity assumption

A tempting but invalid proof strategy would be to claim that a larger oracle family always produces a larger reflected value. Reflected programs can inspect oracle answers and branch, so output need not be monotone. The proposal follows the existing repository discipline instead:

1. progressivity exposes a concrete seed value inside each transformed family;
2. splicing exposes a concrete selected child;
3. strictness follows a fixed child-zero path;
4. all final comparisons compose these explicit occurrences.

This is why the proofs remain valid for arbitrary progressive transformers, not only for a specially monotone object language.

10 How large is the increase?

10.1 What is formally established

The kernel-level claim is exact and modestly stated:

$$\mathrm{contender}_{19} < \mathrm{contender}_{20}\,.$$

More structurally, the formal proof establishes the pointwise sandwich

$$\mathrm{family}_{19}(n) < \mathrm{firstOrder}_{20}(n) < F_{20}(n),$$

where the middle term is a complete first-order diagonal hierarchy with predecessor-sized rank and a schedule syntactically generated from the predecessor schedule by a predecessor-sized hyper-closure.

10.2 Why this is more than another natural rank

Suppose, schematically, that a transformer lift L is an amplification rule on transformers. Contender 19 constructs many such L's and diagonalizes among them, but the rank remains a natural-number recursion *inside the type* $\mathsf{Transformer} \to \mathsf{Transformer}$.

Contender 20 forms operators

$$H : (\mathsf{Transformer} \to \mathsf{Transformer}) \to (\mathsf{Transformer} \to \mathsf{Transformer})$$

and diagonalizes among those. The outer recurrence can use an entire lower lift hierarchy as both its local operator and its seed. In analogy with classical fast-growth constructions, this resembles moving from iterating a fixed normal function to iterating a closure operator that itself manufactures whole function hierarchies. The analogy is heuristic; the Lean theorem does not assign a canonical ordinal index to these operators.

10.3 Why no exact ordinal calibration is claimed

The BusyLean construction combines several ingredients whose interaction is not captured by a standard published hierarchy:

- a strongly normalizing large-Veblen universe ladder;
- dynamic oracle families;
- W-tree schedules generated from language-defined trees;
- transformer-level autonomous closure;
- first- and second-order diagonal reuse;
- ranks and schedules numerically controlled by the predecessor family.

Assigning a proof-theoretic ordinal would require a lower-bound interpretation and a matching upper-bound analysis for this precise higher-order reflection algebra. Any simple expression such as "f_α for this named α" would currently be conjectural. The article therefore makes only the robust structural comparison supported by the source.

10.4 A useful lower-bound reading

Even without an ordinal label, the formal intermediate witness yields a clear lower-bound narrative. At input n, $F_{20}(n)$ exceeds:

1. the complete predecessor family value `family19 n`;
2. one application of a predecessor-sized `diagonalLift` hierarchy to the large-Veblen reflection transformer;
3. that whole first-order layer after it has been installed as the base of a predecessor-sized second-order diagonal hierarchy.

Thus the margin is internal and compositional, not merely a consequence of the outer definition `F20 (F20 42)`.

11 Lean engineering considerations

11.1 Avoiding enormous reduction

The main practical danger in BusyLean proofs is asking the elaborator or simplifier to compare terms by normalizing a contender-sized expression. The new modules avoid that pattern.

- No proof uses `simp [F20]`, `decide`, or arithmetic evaluation on `family19`.
- The final theorem uses only previously proved strict inequalities and `Nat.lt_trans`.
- Definitions are unfolded only at small algebraic interfaces such as `spliceLifts` and one W-tree constructor equation.
- The smoke test contains `#check` and `#print axioms`, never `#eval` or `#reduce`.

11.2 Expected compilation profile

The new code is about four hundred lines including documentation and smoke checks. Its proofs are generic and recurse only over theorem parameters, not concrete gigantic ranks. The concrete numeral `rank20 n` is passed as an opaque argument to already compiled recursive definitions. Accordingly, the expected elaboration cost is governed by the size of the proof terms, not by the value of the contender.

This expectation must still be tested on Lean 4.34.0. The repository's timing rules make build performance part of acceptance.

11.3 Potential elaboration sensitivities

Three local points deserve attention during the first build.

1. Lean must accept the structurally recursive theorem equations for `stageHyperLift_good` and `base_lt_stageHyperLift`. They mirror accepted equations in `HyperReflectionFacts`.

2. One `change` statement relies on definitional reduction of `spliceLifts`, `spliceTransformers`, and the node equation of `stageHyperLift`. If reducibility settings interfere, replace it with explicit local `rfl` equalities.

3. Implicit inference must resolve the nested property predicates. All applications are fully staged enough that any repair should be a local type annotation, not a mathematical change.

12 Integration and validation

12.1 Files to add

Path	Purpose
`BusyLean/SecondOrderReflection.lean`	second-order definitions
`BusyLean/SecondOrderReflectionFacts.lean`	generic goodness and strictness proofs
`BusyLean/Contender20.lean`	schedule, ranks, F_{20}, and closed contender
`BusyLean/Comparison20.lean`	pointwise and closed comparison theorems
`test/SecondOrderReflectionSmoke.lean`	API and axiom smoke checks

Add the four library imports to `BusyLean.lean`. A ready-to-apply patch is included as `busylean_contender20.patch`.

12.2 Required build commands

```
lake build BusyLean.Comparison20
lake env lean test/SecondOrderReflectionSmoke.lean
lake build
lake env lean test/Audit.lean
```

The first command gives the focused kernel check. The full build catches import cycles, naming collisions, and project-wide timing regressions. The existing audit should be extended if it enumerates accepted contender names explicitly.

12.3 Validation actually completed

The package-generation environment completed the following work:

1. audited the incumbent source interfaces and current tree identifier;
2. generated the Lean overlay and patch;
3. ran the static forbidden-marker and declaration audit;
4. compiled this article to PDF;
5. rendered the PDF to images and inspected the pages;
6. generated package checksums.

A Lean executable was not installed, and network installation of the pinned 4.34.0 binary was unavailable. Therefore no statement in this article should be read as claiming that the supplied overlay has already passed a kernel build. The mathematics is reduced to local generic lemmas, but the build commands above remain mandatory.

13 Further research

13.1 Third and higher operator levels

The next literal step is to define

$$\mathsf{UltraLift} = \mathsf{HyperLift} \to \mathsf{HyperLift}$$

and repeat the same construction. Hand-writing every finite level is possible but conceptually unsatisfactory. A more valuable project is a dependent family of carriers

$$A_0 = \mathsf{Transformer}, \qquad A_{k+1} = A_k \to A_k,$$

together with recursively defined goodness, splicing, tree stages, and strict inflationarity. The challenge is to package the varying types so that a runtime-selected finite level can still be collapsed to a natural number.

13.2 Transfinite hyper-ranks

Current ranks are natural numbers, although each rank can contain enormous tree schedules. One could index hyper-ranks by the already formalized large-Veblen order. A successful design would require a well-founded recursion whose successor/limit behavior preserves the explicit-witness strictness proofs. This may be a cleaner route to transfinite operator order than extending the object-language atom ladder again.

13.3 Combining operator order with stronger ordinal atoms

Second-order hyperreflection is orthogonal to the choice of `lTransformer`. If BusyLean later formalizes a Bachmann–Howard, Takeuti–Feferman, or stronger well-founded atom order, the new reflection transformer can be substituted beneath `baseLift20`. The second-order library should require no change.

13.4 Ordinal calibration

A serious calibration program should separate four questions.

1. What standard hierarchy embeds into one progressive reflection transformer over the large-Veblen ladder?
2. What closure corresponds to `liftTransformer` and `diagonalLift`?
3. What new closure is introduced by `liftLift` and `diagonalHyperLift`?
4. Can matching upper bounds be proved by assigning ordinal diagrams to transformer and hyperlift terms?

Without both lower and upper bounds, named-ordinal estimates should remain explicitly heuristic.

13.5 A certified Lean-kernel reflection contender

Longer term, BusyLean could internalize a finite fragment of Lean's kernel or a small dependent type theory strong enough to verify the existing contender modules. A bounded maximum over certified closed programs would subsume many hand-designed object languages. The key research problem is preserving constructive executability while avoiding circular soundness claims.

13.6 Performance instrumentation

The repository would benefit from a standard benchmark script recording:

- clean focused build time for each comparison module;
- maximum resident memory;
- axiom reports for contender and comparison theorem;
- source scan for forbidden constructs;
- proof-term size and imported declaration count.

Such data would make future proposals easier to compare on engineering quality, not only on mathematical ambition.

14 Conclusion

Contender 20 advances BusyLean along the most direct structural axis left open by Contender 19. The incumbent already has a powerful object language and a first-order diagonal hierarchy of transformer lifts. The proposal does not merely lengthen that natural rank. It promotes whole lift hierarchies to the objects manipulated by a new hyperlift level, proves that the corresponding second-order diagonal ranks preserve the exact strict-growth invariant needed by the contest, and embeds a complete predecessor-sized first-order layer as an explicit lower witness.

The resulting formal chain is simple to state:

$$\mathrm{family}_{19}(n) < \mathrm{firstOrder}_{20}(n) < F_{20}(n),$$

and it yields

$$\mathrm{contender}_{19} < \mathrm{contender}_{20}\,.$$

All definitions are structural and intended to remain within BusyLean's constructive rules. The remaining task before acceptance is concrete rather than conceptual: build the overlay with Lean 4.34.0, inspect its axiom report, and verify that focused and full builds remain inside the repository's timing budget.

A Public API summary

Declaration	Role
`HyperLift`	endomorphism of transformer lifts
`spliceLifts`	countable coordinate-selectable splice of lifts
`stageHyperLift`	W-tree traversal with a local hyperlift
`liftLift`	autonomous rank-zero second-order lift
`diagonalHyperLift`	natural-rank second-order diagonal hierarchy
`GoodLift`	preservation plus strict inflationarity
`PreservesGoodLifts`	hyperlift closure invariant
`StrictlyLiftInflationary`	strict comparison at lift level
`diagonalHyperLift_ preservesGoodLifts`	goodness of every rank
`diagonalHyperLift_ strictlyLiftInflationary`	strictness of every rank
`diagonalHyperLift_rank_lt_succ`	adjacent-rank strictness
`family19`	pointwise predecessor family
`schedule20`	predecessor-sized autonomous schedule
`rank20`	predecessor-sized natural rank
`baseLift20`	embedded first-order diagonal lift
`firstOrder20`	numerical value of the embedded first-order layer
`F20`	second-order contender family
`contender_20`	proposed closed BusyLean contender
`family19_lt_firstOrder20`	first strict witness
`firstOrder20_lt_F20`	second strict witness
`contender_19_lt_contender_20`	required closed comparison

B Static audit scope

The included Python audit is intentionally small and reviewable. It performs textual checks after stripping Lean comments, confirms expected declaration markers, verifies critical import edges, and writes hashes and results to JSON. It cannot establish parsing, elaboration, termination, or theorem correctness. Only Lean's kernel can do that. Its purpose is to prevent accidental delivery of unfinished or explicitly nonconstructive source and to make package contents reproducible.

C Selected references

References

[1] BusyLean contributors. *BusyLean: name the biggest number, in Lean.* https://github.com/VladimirReshetnikov/BusyLean, 2026.

[2] Cody Roux. *name-the-biggest-number.* https://github.com/codyroux/name-the-biggest-number.

[3] Scott Aaronson. *Who Can Name the Bigger Number?* https://www.scottaaronson.com/writings/bignumbers.html.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pages 625–635. Springer, 2021.

[5] Kurt Schütte. *Proof Theory.* Springer, 1977.

[6] Wolfram Pohlers. *Proof Theory: The First Step into Impredicativity.* Springer, 2009.